Final Project: Pick a Project

Computer Science - Week 10

Oct 3, 2026 - Version 0.0.2

Please make sure to your UD **email** AND **name** is below. Unless your UD emails are included in this table, then you will not earn any points for this assignment when it is graded! **Every person should submit their own individual worksheet**.

Talk about each of the questions with a friend or our LLM (or both), but **make sure the words you enter below are yours** (with the obvious exceptions notwithstanding). Brainstorming is best done in full sentences, and the main goal here is brainstorming. It's worth reviewing the information in this series of pages in the Drafter documentation to gain a better understanding of the process.

Role	Name	Email
Developer (You)	Robbie Voso	rvoso@udel.edu
Friend (optional)	Logan Gagliardo	logangag@udel.edu

Part 1: The Goal & Prior Implementations

1. **Identify the Goal:** What specific goal are you trying to address? Think about your passions, hobbies, and academic interests. The latter are an excellent candidate to focus on. Some majors have suggestions listed below: these were provided by faculty in those departments. Other majors could consider a focus tied to a big final assignment in one of their recent courses in their major.
    - **Biomedical Engineering** majors may consider one of the following:
        - If you have started (or completed) the Cell/Tissue Design final project in BMEG100, design an organoid, which involves combining a cell type, biomaterial, and testing method. Have the website include a description and diagram of your design concept, a testing method, a needs statement, and an impact statement relevant to their organoid. Also include a link to a paper that supports your idea and a paragraph describing how that paper supports your design concept.

        - Otherwise, create a website related to your BMEG100 Lab Report 1, *Cytoskeletal and Nuclear Staining Lab Report Guidelines*, linked here (University of Delaware account required to access).
    - **Chemistry** and **Chemical Engineering** majors may consider creating an automated chemistry assistant that takes balanced reaction equations, starting reactant masses, and a harvested product amount to determine the limiting reactant, theoretical yield, and percent yield based on principles from Chemistry 2e (OpenStax) — Chapter 4: Chemical Reactions and Stoichiometry. To avoid manually compiling atomic mass tables or writing custom string tokenizers, use the ChemPy library to parse molecular formulas into exact molecular weights and verify reaction stoichiometry against compound data from the NIH PubChem Database. The program outputs an organized breakdown of limiting reagent status, product yields, and remaining masses of unreacted species. Also use the heat of the reaction to determine if it is exothermic or endothermic, and compute the heat flow per mole of the product.
    - Majors in the **Electric & Computer Engineering** department may consider a steganography project that hides messages in images, and then retrieves those messages from uploaded images.
    - **Mechanical Engineering** majors may consider a project that somehow uses both the NumPy and SciPy Python libraries.

Make a simplified simulation of energy market to show effect of building new renewable/non-renewable generation, building data centers

2. **Prior Implementations:** Search the internet. What existing tools or websites currently attempt to address it?

- PyPSA (Python for Power System Analysis): In depth energy market simulation allowing user to build transmission, generation, load and simulate a day

3. **Your Implementation:** Briefly describe the legitimate purpose or functionality of your site. How would you want this web application to be used?

- *User is able to make decisions on planning energy grid, run simulated day/year to see if grid withstands* - *Helps user understand trade-offs between intermittent/traditional energy sources, impact of building large energy users (ie data centers)*

Part 2: The Input-Process-Output (IPO) Model

Think about the fundamental flow of your program.

1. **The Core Goal:** Every program has three main phases: taking input, processing data, and presenting output. In one sentence, what is the ultimate goal your program will accomplish for the user? Be specific and use CS1 concepts.

- The program will input their decisions on how they will manage the electric grid, and output whether or not their grid successfully withstood under pressure

2. Input: What specific information will the user need to provide to your program? (e.g., a city name, a numerical budget, a list of hobbies).

- The user will be presented with options for what they can build (ie a 200 MW solar farm or a data center that uses 50 MW).

3. Processing: At a high level, how will the computer manipulate or evaluate that data? (e.g., calculate the average, filter out options that are too expensive, compare the input against a database).

- The state will contain the total generating capacity and load of the grid
- When the user inputs their choices, these will be added to the state
- A “day” will then be simulated: factors like solar generation will be randomized and a function will check if the generation is sufficient to cover the load

4. Output: What will the user get back? (e.g., a recommendation, a calculated safety factor, a formatted schedule).

- The user will get a message telling them whether the grid they constructed successfully withstood

Part 3: Data Abstraction

Identify the "nouns" of your problem.

1. **Representing Reality:** Abstraction is the process of representing the real world inside a computer by ignoring unnecessary details. What are the core "things" (nouns) your program needs to track? (e.g., *If building a recipe app, the nouns are "Recipe" and "Ingredient"*).

- Generation

- Load
- Emissions
- Cost

2. **Identifying Properties:** For the most important "thing" you listed above, what 3 to 4 specific details do you actually need to solve your problem, and what details will you abstract away?
    - *Details I need:* (e.g., *Ingredient name, weight, cost*)
    - *Details I will ignore:* (e.g., *Ingredient color, brand, expiration date*)

Generation:
- I will need: Generation type (solar/wind/gas), capacity (MW), fuel cost
- I will ignore: Material, color, size

Part 4: Challenges, AI Strategy, & Goals

1. **Difficulties:** What parts of this project seem the most difficult or technically complex?

The most difficult part will be modeling real world grid conditions in a realistic way

2. **Coding Strategy:** If you will be using an LLM to help code, how will you ensure you understand the generated code and avoid "hallucinations" or "shallow learning" (copying without understanding)? If you won't be using an LLM to help code, this project might take about 4 hours a week to complete, so list each of the days and times in which you work on the project.

I will tell the AI to use concepts that I am familiar with and avoid complex methods. I will also have it code in chunks, explaining each chunk as it generates and I will test each chunk as I put the program together.

3. **Goals:** What is your ultimate goal for this project? (e.g., *Impress the graders with a creative UI, build something I can use in my other classes, learn Drafter deeply*).

The goal is to build something that I can talk about to employers to show my understanding of energy markets.

Part 5: Research & Brainstorming Conversation

Have a conversation with a Generative AI tool (like ChatGPT or Claude) OR a friend about your project idea. Do not ask for code. Instead, use the AI/friend as a sounding board to research your problem.

1. **The Initial Prompt/Question:** Ask the AI or your friend to help you brainstorm. (e.g., *"I want to build a simple web tool to help students track their study hours. What are 3 common problems students face when tracking their time, and what are 3 simple features a basic tool should have?"*). Paste your prompt below:

I want to build a web-based energy market simulation game using python, where the user makes decisions to build different types of power plants, data centers, etc. It will then run the user's grid against a day to see if the grid withstands. What are 3 aspects of the grid that I should incorporate, and what are 3 simple features a basic tool should have?

2. **The Response:** Briefly summarize the best ideas the AI or your friend suggested. What existing solutions or prior apps did they mention?

The AI suggested that I incorporate:

- 24 hour "duck" shaped demand curve that generation must meet
- Realistic availability of generation sources (ie solar and wind not always available)
- Reserve margins: Track how much reserves are available at each hour
- "Build" menu where user makes decisions to build, "run" button to check if it withstands

3. **The Pushback:** You must reply to the AI/friend to narrow the scope when it's over-ambitious. Pick the most complex feature they suggested and explain why it is too complicated for an introductory programming project. Ask them for a simpler alternative. *Paste your response here:*

Given that this is an introductory programming project, it may be too ambitious to incorporate marginal costs and "economic dispatch" like you mentioned. How can I simplify this so it still gets the point across without bringing in complex market dynamics.

4. **Reflection:** Based on this conversation, how has your project idea changed or become more focused?

After the conversation, I will focus less on market dynamics and pricing and make it a more simple comparison of supply vs demand to determine if there is enough supply.

Part 6: Resource Planning

1. What resources will you use if you get stuck?

I will use drafter documentation, course resources, LLMs, and energy-related online resources if I get stuck.

2. If your LLM provides code that causes a Drafter or Bakery error, or if you run into a problem during deployment, what is your step-by-step debugging plan to resolve it?

I will incorporate numerous unit tests in order to find where the error stems from. I will then test the user-facing interface with various inputs to see how the program holds up.

Part 7: LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

I want to build a web-based energy market simulation game using python, where the user makes decisions to build different types of power plants, data centers, etc. It will then run the user's grid against a day to see if the grid withstands. What are 3 aspects of the grid that I should

incorporate, and what are 3 simple features a basic tool should have?

Here's my ranked take, with the grid aspects first.

3 grid aspects to model

1. **Hourly supply and demand balance.** Run 24 hours where the player's generation has to cover load every hour. Demand should have a realistic daily shape (morning ramp, evening peak), and data centers should add a flat, high baseload on top. This is the core of whether the grid "survives."
2. **Generator characteristics and dispatch order.** Each plant type needs capacity, marginal cost, and availability. Solar only produces midday, wind is variable, gas is dispatchable and pricey, nuclear is cheap but inflexible. Dispatch cheapest first (merit order), and the last plant needed sets the price. This is where the market side comes alive.
3. **Reserve margin and reliability.** Track how much spare capacity exists above demand each hour, and score unserved energy (blackout hours) as a failure. Add one random stress event like a heat wave demand spike or a plant outage so players can't just build to the average.

3 simple features for a basic tool

1. **Build menu with a budget.** Let the player pick plant types and quantities, each with a cost and capacity, under a fixed capital budget. This forces tradeoffs.
2. **Run-the-day button with a results chart.** One click simulates 24 hours and plots demand against each generation source stacked by hour. Seeing the gap where supply falls short is the payoff moment.
3. **Scorecard.** Show total cost, average price, reliability (hours or MWh unserved), and maybe emissions. A single summary makes it replayable and easy to compare attempts.

For the stack, Flask or Drafter for the web layer plus pandas and matplotlib or Plotly for the simulation and charts keeps it simple. If you want to start even smaller, build only aspects 1 and 2 with three plant types first, then add reserve margin and events once the dispatch loop works.

Given that this is an introductory programming project, it may be too ambitious to incorporate marginal costs and "economic dispatch" like you mentioned. How can I simplify this so it still gets the point across without bringing in complex market dynamics.

Settling on the final reply style

Cut the market side completely and keep it as a capacity-versus-demand game. The budget still gives you the tradeoffs, so you don't lose the point.

What to simplify, ranked by how much complexity it removes

1. **Drop prices and dispatch order.** Every plant just produces its output each hour and you add them up. No merit order, no marginal cost, no price setting.
2. **Use a hardcoded output profile per plant type.** Each type gets a list of 24 numbers representing the fraction of its capacity available each hour. Solar is zero at night and peaks midday, wind is a bumpy preset, and gas and nuclear are a flat 1.0. This captures why a mix matters without any real modeling.
3. **Hardcode demand as a list of 24 numbers.** Make data centers a flat number added to every hour. No forecasting needed.
4. **Make the budget the only economic piece.** Each plant has a build cost and a capacity, and the player can't exceed the budget. That's the whole "market" element.
5. **Keep one stress event as a simple multiplier.** A heat wave multiplies demand by 1.2 for the afternoon hours, or a plant outage zeroes out one plant. Pick one and let the player choose or randomize it.

The core loop then looks like this

python

Python

```
for hour in range(24):
    supply = sum(count[p] * capacity[p] * profile[p][hour] for p in
plants)
    shortfall = max(0, demand[hour] - supply)
```

Count the hours where the shortfall is above zero and that's your reliability score. Total build cost is your cost score. Those two numbers plus one chart of supply against demand is a complete game.

If you have time left at the end, the easiest add-on is a reserve margin readout, which is just supply minus demand per hour. It's one more line and it teaches the concept without any extra mechanics.

Part 8: Submit

In Google docs, with this document open, please click on *File* > *Download* > *Microsoft Word (.docx)*, and make sure you put that download somewhere you can find it again.

Then, upload it to the Canvas assignment about selecting or brainstorming your final project.

When you're done with this assignment, review this page of the Drafter documentation to better prepare for the next step.